

os第二次理论作业

1.

动态内存分配需要对内存分区进行管理，一般使用位图和空闲链表两种方法。128MB的内存以 n 字节为单元分配，对于链表，假设内存中数据段和空闲区交替排列，长度均为64KB。并假设链表中的每个节点需要记录32位的内存地址信息、16位长度信息和16位下一节点域信息。这两种方法分别需要多少字节的存储空间？哪种方法更好？

- 对于位图：由于 $128MB = 2^{27}B$ ，故共有 $2^{27}/n$ 个单元，每一个单元需要一个标记为，故共需要 $2^{27}/n$ 位，即 $(2^{24}/n)B$ 的存储空间。
- 对于链表：需要 $128M/64K = 2^{11}$ 个节点，每个节点 $32 + 16 + 16 = 64bit = 8B$ ，共需要 $2^{11} \times 8 = 2^{14}B$ 的存储空间。
- n 比较小时，链表更省空间， n 比较大时，位图更省空间。临界点是 $n = 1KB$

2.

在一个交换系统中，按内存地址排列的空闲区大小是：10KB、4KB、20KB、18KB、7KB、9KB、12KB和15KB。对于连续的段请求：12KB、10KB、9KB。使用FirstFit、BestFit、WorstFit和NextFit将找出哪些空闲区？

- FirstFit:
 - 12KB：找到空闲是20KB，分割完后：10、4、8、18、7、9、12、15
 - 10KB：找到空闲是10KB，分割完后：4、8、18、7、9、12、15
 - 9KB：找到空闲是18KB，分割完后：4、8、9、7、9、12、15
- BestFit:
 - 12KB：找到空闲是12KB，分割完后：10、4、20、18、7、9、15
 - 10KB：找到空闲是10KB，分割完后：4、20、18、7、8、15
 - 9KB：找到空闲是15KB，分割完后：4、20、18、7、8、6
- WorstFit:
 - 12KB：找到空闲是20KB，分割完后：10、4、8、18、7、9、12、15
 - 10KB：找到空闲是18KB，分割完后：10、4、8、8、7、9、12、15
 - 9KB：找到空闲是15KB，分割完后：10、4、8、8、7、9、12、6
- NextFit:
 - 12KB：找到空闲是20KB，分割完后：10、4、8、18、7、9、12、15
 - 10KB：找到空闲是18KB，分割完后：10、4、8、8、7、9、12、15
 - 9KB：找到空闲是9KB，分割完后：10、4、8、8、7、12、6

3.

解释逻辑地址、物理地址、地址映射，并举例说明。

- 逻辑地址：程序视角中的地址，由编译器生成，与物理内存无关。程序通过逻辑地址访问内存，它属于进程的虚拟地址空间。举例：在C语言中，`int *ptr = (int *)0x1000;` 这里的0x1000是逻辑地址。

- 物理地址：内存硬件中的实际地址，对应物理内存芯片上的存储单元。全局唯一，由内存控制器直接使用
通常对程序员不可见，由操作系统管理。举例：若内存条的物理地址范围为0x0000至0xFFFF，0x5234即为一个物理地址。
- 地址映射：将逻辑地址转换为物理地址的过程，由内存管理单元（MMU）和操作系统协同完成。
核心机制：
分段：逻辑地址 = 段选择符 + 段内偏移 → 物理地址 = 段基址 + 偏移
分页：逻辑地址 = 页号 + 页内偏移 → 通过页表查物理页框号 → 物理地址 = 页框号 + 偏移
举例：分页机制下的地址映射。若页大小 = 4KB (0x1000)，逻辑地址0x1234，页表中页号0x1对应物理页框号0x5。页号 = $0x1234 / 0x1000 = 0x1$ ，页内偏移 = $0x1234 \% 0x1000 = 0x234$ ，查页表：页号0x1 → 物理页框号0x5，物理地址 = $0x5 * 0x1000 + 0x234 = 0x5234$

4.

解释页式（段式）存储管理中为什么要设置页（段）表和快表，简述页式（段式）地址转换过程。

- 页式存储管理：
 - 页表：记录逻辑页号到物理页框的映射关系，实现虚拟地址到物理地址的转换。
 - 快表：加速转换，缓存常用页表项，减少访问内存页表的次数。硬件优化，并行查询TLB和页表，提高地址转换效率。
 - 转换过程：
 1. 分解逻辑地址：逻辑地址 = 页号（高位） + 页内偏移（低位）
(例：逻辑地址0x1234，页大小4KB → 页号=0x1，偏移=0x234)
 2. 查页表：用页号检索页表，获取对应的物理页框号
(页表中页号0x1 → 物理页框号0x5)
 3. 合成物理地址：物理地址 = 物理页框号 × 页大小 + 页内偏移
(例： $0x5 \times 0x1000 + 0x234 = 0x5234$)
- 段式存储管理：
 - 段表：记录每段的基地址、长度及权限（如代码段、数据段）
 - 快表：缓存常用段描述符，加速段基址查询
 - 转换过程
 1. 分解逻辑地址：逻辑地址 = 段号 + 段内偏移
(例：段号=2，偏移=0x500)
 2. 查段表：用段号检索段表，获取段基地址和段长度
(段表中段号2 → 基址=0x8000，长度=0x2000)
 3. 验证偏移：检查偏移是否在段长度内
($0x500 < 0x2000 \rightarrow$ 有效)
 4. 合成物理地址：物理地址 = 段基地址 + 段内偏移
(例： $0x8000 + 0x500 = 0x8500$)

5.

叙述缺页中断的处理流程。

1. 触发缺页中断

- 当CPU访问虚拟地址时，内存管理单元（MMU）通过页表查询发现目标页的存在位（Present Bit）为0，或检测到权限不符（如写只读页），触发缺页中断。

2. 保存上下文并进入内核态

- CPU暂停当前进程，保存其上下文（如寄存器状态），切换至内核态，执行操作系统的缺页中断处理程序。

3. 检查虚拟地址合法性

- 操作系统检查触发缺页的虚拟地址是否属于进程的合法地址空间（如代码段、堆、栈等）。若地址非法（如越界访问），触发段错误（如发送SIGSEGV信号）终止进程。

4. 验证访问权限

- 若地址合法但权限不符（如写只读页），进一步判断是否为写时复制（Copy-on-Write）场景。若是，则为进程分配新物理页，复制原内容，更新页表项为可写；否则报错。

5. 分配物理页帧

- 若有空闲物理页：直接使用。
- 若无空闲页：调用页面置换算法（如LRU、时钟算法）选择牺牲页。若牺牲页被修改过（脏页），需先写回磁盘。

6. 加载缺失页面到内存

- 根据页面类型确定数据源：
 - 匿名页（如堆/栈）：从交换空间（Swap）读取。
 - 文件映射页：从对应文件块读取（如代码段或mmap文件）。

7. 更新页表及TLB

- 将虚拟页映射到新物理页帧，设置页表项的存在位、权限位等。
- 使TLB中对应条目失效，或更新TLB，确保后续访问直接命中。

8. 恢复进程执行

- 恢复进程上下文，重新执行引发缺页的指令。此时虚拟地址已映射到物理内存，指令正常执行。

6.

假设一个机器有38位的虚拟地址和32位的物理地址。

(1) 与一级页表相比，多级页表的主要优点是什么？

(2) 如果使用二级页表，页面大小为16KB，每个页表项有4个字节。应该为虚拟地址中的第一级和第二级页表域各分配多少位？

- 节省内存空间：一级页表需要为整个虚拟地址空间预先分配连续内存，即使大部分区域未使用。多级页表通过分层结构，仅分配实际使用的页表项，避免稀疏地址空间的冗余存储。灵活性与扩展性：多级页表支持动态扩展，按需分配二级页表，适合大型虚拟地址空间（如38位虚拟地址）。物理内存高效利用：未使用的二级页表无需驻留内存，减少物理内存占用。
- 计算过程如下：
 - 每页容纳页表项：页面大小 $16KB = 2^{14}B$ ，页内偏移14位。每页容纳表项数 $2^{14}/4 = 2^{12}$ ，故每一级页表项索引12位。
 - 虚拟地址划分：38位（虚拟地址）= 一级页表索引 + 二级页表索引 + 页内偏移。对应： $38 = 12（一级）+ 12（二级）+ 14（偏移）$

7.

假设页面的访问存在一定的周期性循环，但周期之间会随机出现一些页面的访问。例如0,1,2,...,511,431,0,1,2,...511,332,0,1,2,...,511等。请思考：

(1) LRU、FIFO和Clock算法的效果如何？

(2) 如果有500个页框，能否设计一个优于LRU、FIFO和Clock的算法？

- LRU、FIFO和Clock算法的效果分析

在周期性循环访问（如0到511顺序访问）且页框不足的情况下，LRU、FIFO和Clock算法的表现均不理想：

- LRU：每个周期访问到第500个页面时，内存已满，后续的12个页面（500到511）每次访问都会置换掉最久未使用的页面（如0、1等）。这导致第一个周期内后12次访问和随机页面访问和后续周期均产生缺页，缺页率较高。
- FIFO：与LRU类似，置换最早进入的页面，导致每个周期后半部分的页面访问频繁触发缺页。
- Clock：近似LRU，但因顺序访问导致引用位频繁置1，最终表现接近FIFO，仍无法避免周期性缺页。

共性缺陷：无法识别周期性模式，无法主动保留周期内重复访问的页面，导致每个周期后半部分及随机访问的页面频繁缺页。

- 考虑一个能够识别周期的算法，以访问0作为周期开始。保留0-499页面，这样每个周期只会出现13次缺页。

8.

一个交换系统通过紧缩技术来清理碎片。如果内存碎片和数据区域是随机分配的。而且假设读写32位内存字需要10nsec. 那么如果紧缩128MB的内存需要多久？简单起见，假设第0个字是碎片的一部分而最高位的字包含了有效的数据。

- $128MB = 32M$ 字
- 每字一次读写消耗： $2 \times 10nsec = 20nsec$
- 若需要移动内存比例为50%： $4 \times 10^6 \times 20nsec = 80 \times 10^6 nsec = 80ms$
- 极端情况：
 - 全部需要移动： $32 \times 10^6 \times 20nsec = 640ms$
 - 只需要移动最高位1字： $1 \times 20nsec = 20nsec$